

Multi-tenancy approach

How the PGR Platform isolates each institution's data — shared database, shared schema, row-level security.

In one line

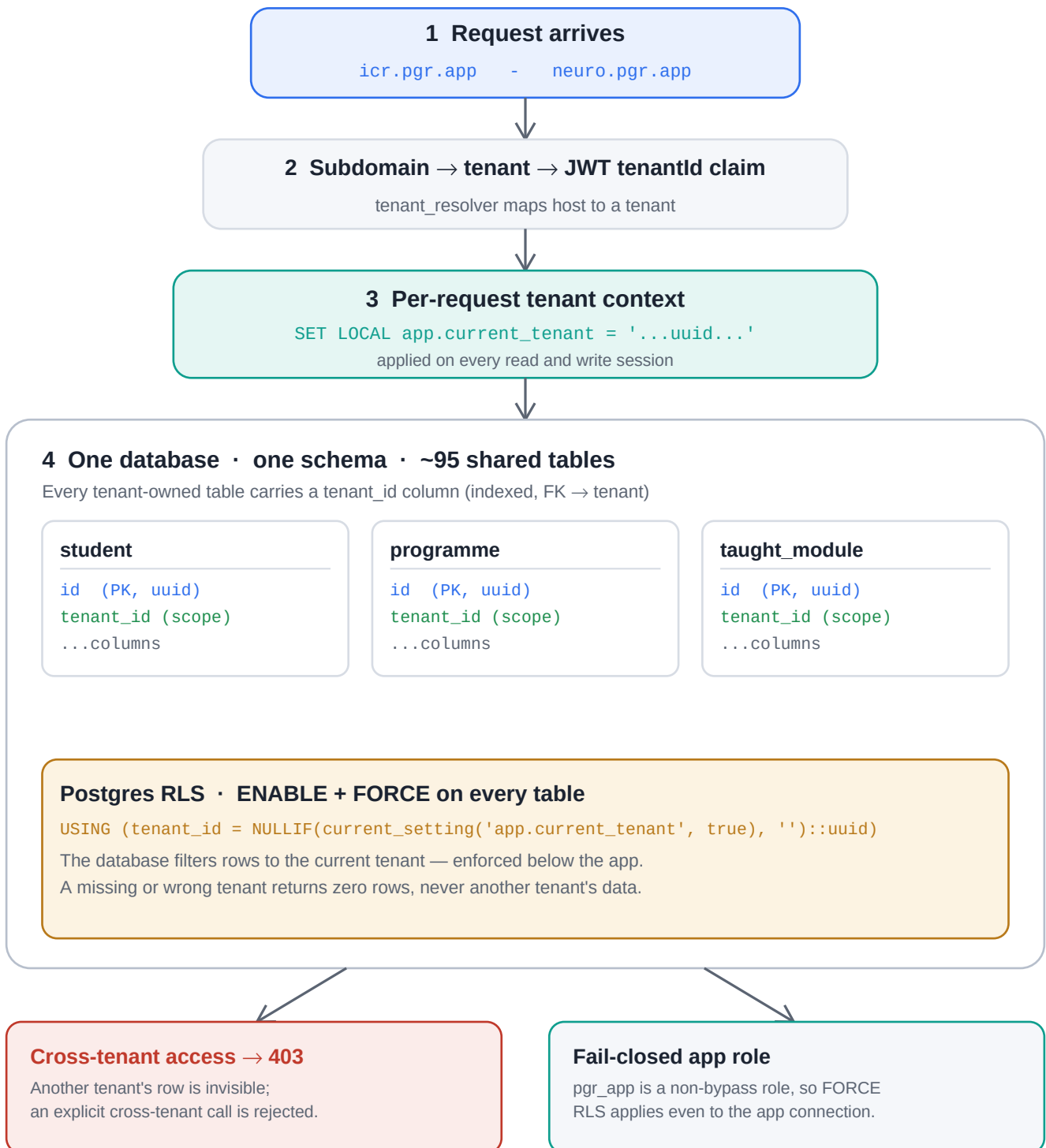
The platform runs a **single database with a single schema**. Every tenant-owned row is **stamped** with a `tenant_id`, and **PostgreSQL Row-Level Security (RLS)** guarantees a query can only ever see the rows of the tenant resolved from the request. Isolation is enforced **inside the database engine**, on every read and write — not by the application remembering to filter, and not by the shape of the primary key.

Clearing up a common assumption

Assumption	Actual design
<code>tenant_id</code> is part of a composite primary key	The primary key stays a single-column UUID id
Isolation comes from the key structure	Isolation comes from RLS policies in the database
Rows are “dumped” keyed by tenant	Rows are stamped with <code>tenant_id</code> ; the DB then refuses non-matching rows

Why this is stronger: because the guarantee is structural to the database rather than to application code, a bug that forgets to add a tenant filter cannot leak data — the engine still returns zero cross-tenant rows.

Request → row-level security flow



How a request is scoped, step by step

1 Request arrives	A request lands on a tenant subdomain, e.g. <code>icr.pgr.app</code> or <code>neuro.pgr.app</code> .
2 Tenant resolution	<code>tenant_resolver</code> maps the host to a tenant; the JWT carries a <code>tenantId</code> claim.
3 Per-request context	On every session — reads included — the app issues <code>SET LOCAL app.current_tenant = ''</code> (wired via the <code>after_begin</code> event).
4 Scoped data access	All ~95 tenant-owned tables have a <code>tenant_id</code> column; RLS (<code>ENABLE + FORCE</code>) filters every query to the current tenant.
5 Outcomes	Cross-tenant access yields zero rows / 403; the fail-closed <code>pgr_app</code> role means <code>FORCE</code> RLS applies even to the application's own connection.

Two safety details worth remembering

- **The NULLIF wrapper.** Pooled connections can revert the `app.current_tenant` GUC to an empty string. Casting `''::uuid` would raise a 500, so the predicate uses `NULLIF(current_setting(...), '')` — empty becomes NULL, the policy is simply permissive-when-unset instead of crashing.
- **Fail-closed application role.** `pgr_app` is a non-bypass role, so `FORCE` RLS applies even to the app's own connection. A code path that forgets to set the tenant returns no rows rather than leaking across tenants.

Where this lives in the codebase

Concern	Location
<code>tenant_id</code> column + write default	<code>TenantMixin</code> — <code>app/db/base.py</code>
host → tenant resolution	<code>tenant_resolver.py</code>
RLS DDL helper	<code>tenant_ddl.py</code>
Fail-closed role provisioning	<code>deploy/provision_app_role.sql</code>
Design + rollout notes	<code>deploy/MULTI_TENANCY.md</code>